Allocation domain decision check

I added and hand-computed a new four-layer case with costs `[3, 0.5, 2, 4]` and minimum allocations of `[1, 1, 2, 1]`. The case is feasible under a `<= 12`, as the minimum allocation for all four layers is `11.5`. Therefore, the remaining `0.5` can remain unspent. Additionally, as demonstrated by the new case, both fractions and whole numbers work.

This implementation only covers whether we or not we can afford each layer's minimum, and proves at least one allowed allocation exists.

My next step would likely to find possible allocations based on a chosen algorithm or formula, though this is outside the scope of the current checkpoint.

The program that does so would need to have predefined rules, such as whether non-integer allocations are allowed and whether the budget can be partly unused. The program should also immediately reject any budgets below the minimum cost before finding possible allocations.

This is a minimum-budget check only, and was computed by hand. It does not establish reproduce the authors' code, experiments, or language model resuslts. The solver implementation remains outside this checkpoint. No Modal compute credits were spent.